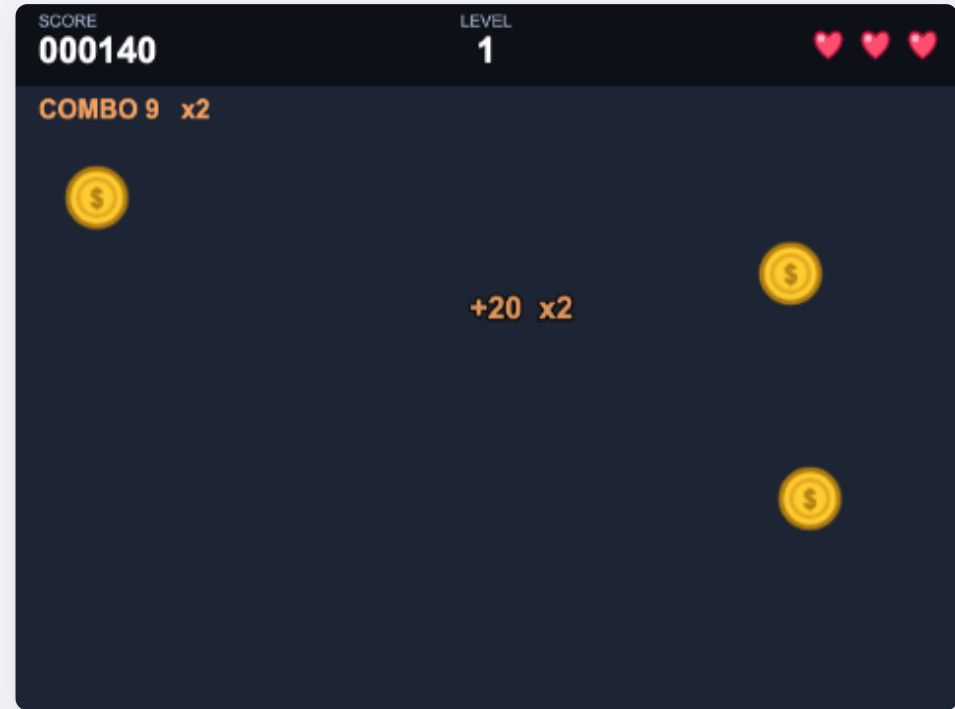


Chapter 6

Scoring

A HUD, a ScoreManager, and high scores that last



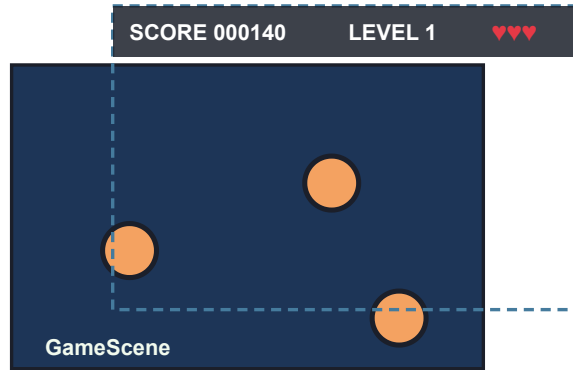
Today

- a **HUD** in a scene of its own, on top of the game
- three ways to tell the HUD something changed
- a **ScoreManager**: tell it what happened; it announces the results
- floating "+10"s and a score that counts up
- high scores saved in the browser - and checked when they come back
- typing initials

A HUD is a scene

Two scenes running at once, one drawn on top of the other

HudScene (see-through, except its bar)



main.ts

```
scene: [TitleScene, GameScene,  
        HudScene, GameOverScene]
```

Later in the list = drawn later = on top.

GameScene

```
this.scene.launch(HUD_SCENE, data)  
starts the HUD beside it, on the next frame
```

```
this.scene.stop(HUD_SCENE)  
takes it away again when the game ends
```

```
this.scene.launch(HUD_SCENE, hudData); // beside this scene, from the next frame  
...  
this.scene.stop(HUD_SCENE);           // when the game ends
```

Why a separate scene?

- **its own camera** - `this.cameras.main.shake(...)` in the game does not shake the score
- draw order = order in the config's `scene` list: later is on top
- the game scene can restart without the HUD rebuilding itself
- one job per class: the game plays, the HUD shows

Emitters

An **event emitter**: `emit` events on it; listen with `on`, `once`, `off`

```
ball.on("pointerdown", ...)    // you have used one already
```

Three that can reach a HUD:

- the game scene's own: `this.events`
- the game's: `this.game.events`
- the registry's: `this.registry.events`

Try it: HUD Events

- click the coin, gem and star (or 1, 2, 3)
- **R** restarts the game scene; **H** restarts the HUD
- watch the "listeners" counts
- after H, why does it say `Gems: ?`



Three routes

```
// 1 - the game scene's own emitter (the HUD needs the scene)
this.events.emit(COINS_CHANGED, this.coins);
this.gameScene.events.on(COINS_CHANGED, this.showCoins, this);

// 2 - the game's emitter (anyone can send, anyone can listen)
this.game.events.emit(GEMS_CHANGED, this.gems);
this.game.events.on(GEMS_CHANGED, this.showGems, this);

// 3 - the registry announces its own changes
this.registry.inc(STARS, 1);
this.registry.events.on(`changedata-${STARS}`, this.onStarsChanged, this);
```

First `set` of a new key sends `setdata`, not `changedata`

News and state

- an event is **news**: only listeners there *at the time* hear it
- a launched HUD starts a frame late - so it also reads the current values

```
this.showCoins(this.gameScene.getCoins()); // route 1: ask the scene  
this.gemsText.setText("Gems: ?");         // route 2: nobody to ask  
this.showStars(this.registry.get(STARS));  // route 3: registry = state + news
```

Clean up

```
this.events.once(Phaser.Scenes.Events.SHUTDOWN, () => {  
  this.gameScene.events.off(COINS_CHANGED, this.showCoins, this);  
  this.game.events.off(GEMS_CHANGED, this.showGems, this);  
  this.registry.events.off(`changedata-${STARS}`, this.onStarsChanged, this);  
});
```

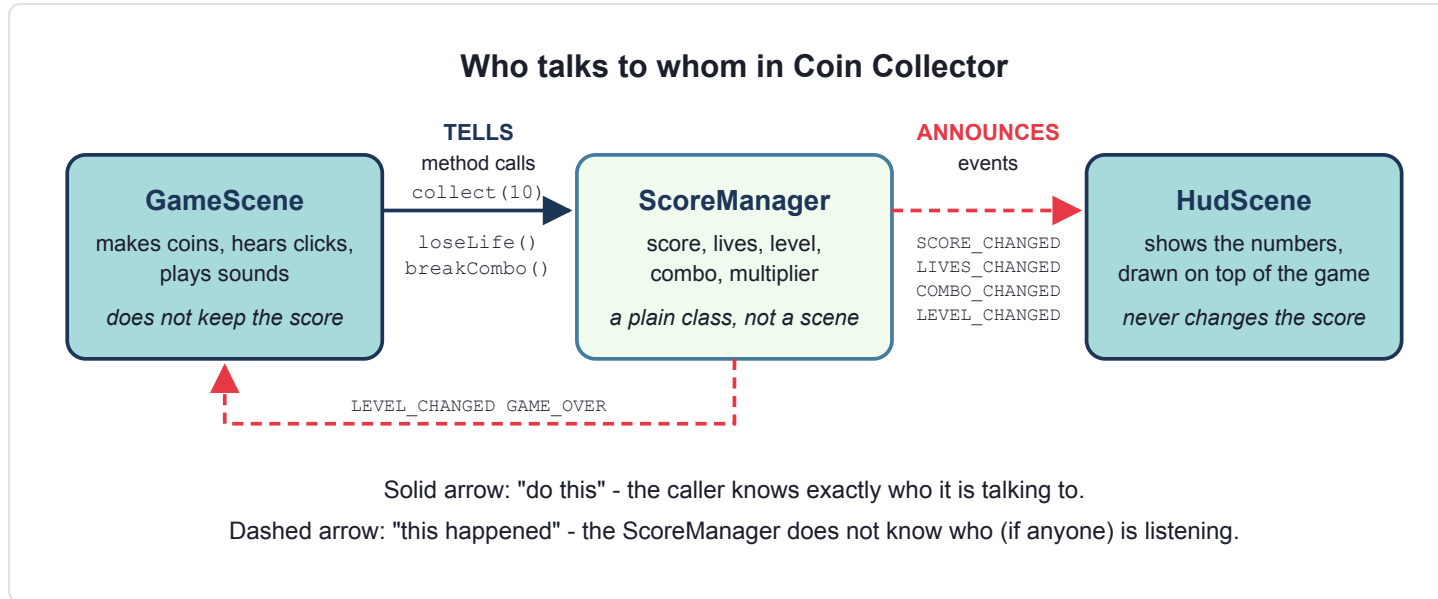
- those emitters **outlive the HUD** - Phaser will not remove these listeners
- forget it: **no error**, the listeners just pile up
- `off` needs the same event, function **and context** as `on`

A ScoreManager

```
export class ScoreManager extends Phaser.Events.EventEmitter {  
  private score = 0;  
  private lives = START_LIVES;  
  ...  
  public collect(basePoints: number): number {  
    this.combo = this.combo + 1;  
    const points = basePoints * this.getMultiplier();  
    this.score = this.score + points;  
    this.emit(SCORE_CHANGED, this.score, points);  
    ...  
  }  
}
```

A plain class: no text, no sounds, no scenes

Tell - announce



The observer pattern - Swing's `addActionListener`

The HUD listens

```
init(data: HudData): void {  
    this.scoreManager = data.scoreManager;  
    ...  
}  
  
create(): void {  
    ...  
    this.showScore(this.scoreManager.getScore());           // now  
    this.scoreManager.on(SCORE_CHANGED, this.countUpTo, this); // and later  
    ...  
}
```

New game? `this.scoreManager = new ScoreManager();` - nothing to reset

Try it: Coin Collector

- 5 in a row raises the multiplier
- 10 pickups = a faster level
- 3 escape = game over
- find every `emit` and every `on`

COIN COLLECTOR



10 points



50 points

Click them before they vanish.
Every 5 in a row raises your multiplier.
Let 3 get away and the game is over.

Best score: 0

Press SPACE to start

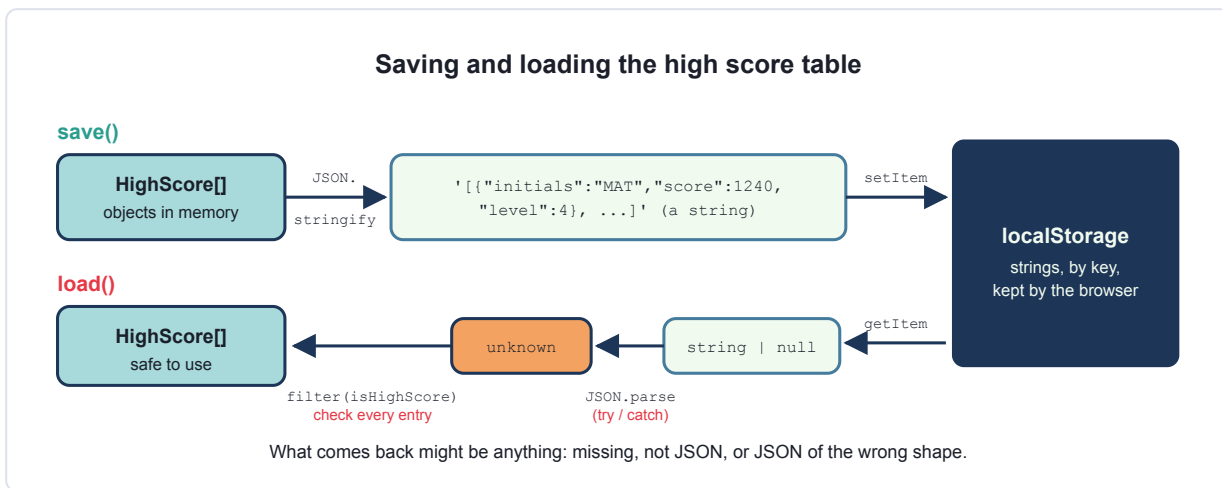
Feel: float and count

```
// FloatingText: rise, fade, then destroy itself
scene.tweens.add({ targets: this, y: y - RISE, alpha: { value: 0, ease: "Cubic.easeIn" },
  duration: DURATION, ease: "Cubic.easeOut", onComplete: () => this.destroy() });

// HudScene.countUpTo(score)
this.countUp?.stop();
this.countUp = this.tweens.addCounter({
  from: this.shownScore, to: score, duration: COUNT_UP_TIME, ease: "Cubic.easeOut",
  onUpdate: (tween) => {
    this.showScore(Math.round(tween.getValue() ?? 0));
  },
});
```

localStorage and JSON

- `setItem(key, text)`, `getItem(key)` (or `null`), `removeItem(key)`
- **strings only** - `JSON.stringify` / `JSON.parse`
- kept **per web site**: every project on `127.0.0.1:8000` shares one - name your key
- each browser has its own (Celbridge's webview is not Chrome)



Never trust what comes back

```
let data: unknown;
try {
  data = JSON.parse(text);
} catch {
  return new HighScoreTable([]);           // not JSON at all
}
if (!Array.isArray(data)) {
  return new HighScoreTable([]);           // JSON, but not a list
}
const entries = data.filter(HighScoreTable.isHighScore);
```

`unknown`: check before use. `isHighScore(value: unknown): value is HighScore` - a **type guard**

Try it: High Score Table

- get onto the table; find the key in the developer tools
- change it to `this is not json` and reload
- what happens to a score that *ties*?

COIN COLLECTOR

HIGH SCORES

1.	ACE	002450	level 7
2.	BOB	001980	level 6
3.	CAT	001620	level 5
4.	DAN	001400	level 5
5.	MAT	001240	level 4
6.	EVE	001130	level 4
7.	FOX	000960	level 4
8.	GUS	000720	level 3
9.	HAL	000540	level 3
10.	IVY	000350	level 2

Press SPACE to start

Click coins before they vanish. 5 in a row raises the multiplier.

Typing initials

```
this.input.keyboard!.on("keydown", (event: KeyboardEvent) => {  
    this.handleKey(event);  
});  
...  
if (LETTER.test(event.key) && this.initials.length < MAX_INITIALS) {  
    this.initials = this.initials + event.key.toUpperCase();  
} else if (event.key === "Backspace") {  
    this.initials = this.initials.slice(0, -1);  
}
```

`event.key` - what the key types. `event.code` - where the key is

Summary

- a HUD is a scene, launched on top; it has its own camera
- emitters: `this.events` , `this.game.events` , `this.registry.events` , your own
- events are news: read the current state too, and clean up on `SHUTDOWN`
- scenes **tell** the ScoreManager; it **announces**
- tweens and counter tweens make scores feel good
- `localStorage` + JSON for scores that last - validated on the way back in

Challenges

1. **A star is born** - a rare 100-point star
2. **Bonus life** - a new level gives a life back; the HUD needs no change
3. **Combo meter** - a bar that fills towards the next multiplier
4. **Wipe the slate** - R, then Y, clears the table
5. **Chasing the record** - HI in the HUD, and a celebration
6. **Easy, normal, hard** - a table for each difficulty

Next: **Chapter 7 - Animations**